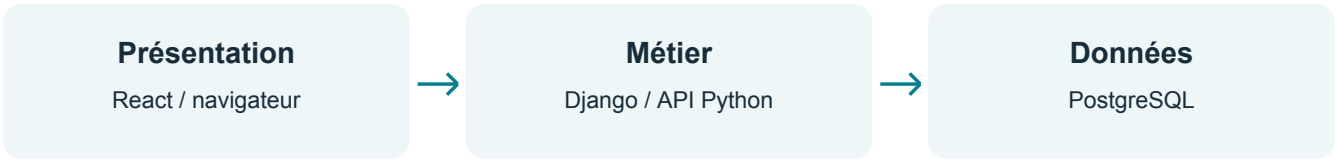
Une application, deux déploiements

Intern

Conception et développement d'une application Web trois-tiers, déploiement sur trois machines virtuelles, conteneurisation et comparaison expérimentale avec Apache JMeter.

Hicham et Omar
Polytech Marseille · Année universitaire 2026-2027



Résumé

Intern accompagne la recherche de stages et le suivi des candidatures. Le projet sépare l'interface, les règles métier et le stockage, puis conserve cette séparation dans deux infrastructures : trois VM VirtualBox et trois services Docker.

Les preuves archivées couvrent le fonctionnement applicatif, les échanges réseau et seize paliers de charge. Dans les essais de référence, Docker utilise moins de mémoire et de disque, mais atteint le catalogue plus tard. Sous charge, les débits sont proches et la latence p95 est plus faible avec Docker dans la plage étudiée.

Limite centrale : aucune saturation serveur n'est observée de 1 à 20 utilisateurs. Le rapport présente les mesures obtenues sans leur attribuer une capacité maximale ni une validation pédagogique non documentée.

Repères de lecture

Pages	Contenu
2-6	Besoin, conception, données, API, interface et tests
7-8	Déploiement VM et conteneurisation
9-10	Comparaison VM/Docker : section de deux pages
11-14	Scénario JMeter, étalonnage, campagnes et analyse
15	Bilan, points à compléter et dossier de remise

Besoin, rôles et périmètre

Le domaine retenu est la recherche de stages en Europe. Un candidat prépare son profil et ses documents, consulte les offres et suit ses dossiers. Le recruteur publie des offres et examine uniquement les candidatures qui lui sont transmises. L'administrateur supervise les comptes et le catalogue.

Acteur	Actions principales	Frontière d'accès
Candidat	Profil, CV, offres, favoris et candidatures	Ses données privées et ses dossiers
Recruteur	Vérification, offres et suivi des candidats reçus	Son espace ; aucun accès général aux CV
Administrateur	Activation des comptes et gestion des référentiels	Administration réservée

Règles et cas limites

Une offre référence une entreprise existante. La source d'une candidature doit appartenir à l'offre visée. Le couple candidat/offre est unique. Le login est unique et le mot de passe est protégé par un hachage salé lent. Un nouveau compte attend l'activation avant de pouvoir se connecter.

Les contrôles couvrent le mot de passe erroné, le compte inactif, l'absence de résultat ou de candidature, le document invalide et l'accès à une ressource tierce. Le partage d'une entreprise entre deux recruteurs ne leur donne pas accès aux mêmes offres privées.

Parcours de référence

Le candidat s'inscrit, attend l'activation, se connecte, complète son profil et son CV, puis crée et consulte une candidature. Le recruteur vérifie son adresse et son entreprise, prépare une offre et consulte les dossiers reçus. L'administrateur valide les demandes dans son interface dédiée.

Adaptation du sujet du cours

Le sujet transcrit dans les consignes porte sur la consultation de notes de cours. Intern transpose les objectifs pédagogiques à un autre domaine : rôles, activation, propriété des ressources et séparation trois-tiers. Les règles R1-R6 du projet sont donc une adaptation, pas une reprise à l'identique des règles scolaires.

À confirmer avant le rendu : l'acceptation de cette adaptation et la validation du jalon Figma ne sont pas établies par les documents consultés.

Sources : analyse/cas_d_usage.md ; docs/consignes/Dev.md ; docs/BACKEND_SPEC.md.

Concevoir les parcours avant le code

Le prototype Figma rassemble les parcours candidat, recruteur et administrateur. L'écran zéro sert de sommaire pour accéder aux écrans principaux et aux états d'erreur ou d'absence de données.

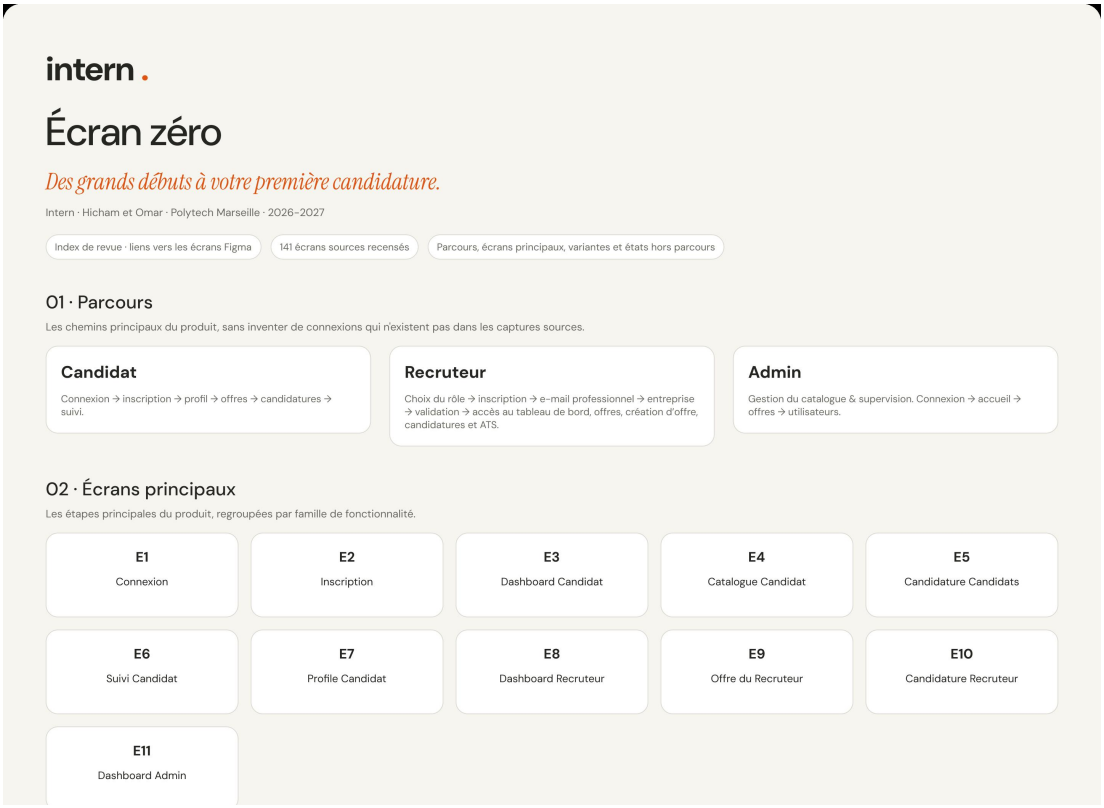


Figure 1. Extrait supérieur de l'écran zéro livré ; la capture complète, plus longue, reste dans maquette/ecran_zero.jpg. Cet extrait montre l'organisation de la revue, pas la vérification de tous les liens interactifs.

Principes d'interaction

La navigation doit rendre visibles les étapes du parcours et permettre d'identifier chargement, liste vide et échec. Les données de démonstration sont fictives mais cohérentes avec les actions proposées. Le rôle est déterminé par le serveur après authentification ; le choix de profil à l'inscription ne confère aucun privilège administratif.

Traçabilité et validation

Les cas d'usage de l'analyse sont rapprochés des écrans attendus. Les identifiants des parcours sont conservés dans le manifest et les liens de consultation dans figma_link.txt. La présence de ces fichiers ne remplace pas l'essai du prototype par un lecteur externe.

Contrôle restant : vérifier les droits de lecture et la navigation complète, puis comparer les écrans réels au prototype. Le cahier de recette conserve ce contrôle sous le test T25, encore à vérifier.

Sources : maquette/README.md ; maquette/figma_link.txt ; maquette/figma_manifest.json ; maquette/ecran_zero.jpg.

Un modèle relationnel cohérent

PostgreSQL conserve comptes, entreprises, offres, profils, documents et candidatures. Le schéma exporté reprend les migrations de l'application ; la vue suivante présente uniquement les relations principales, parmi les 72 tables documentées.

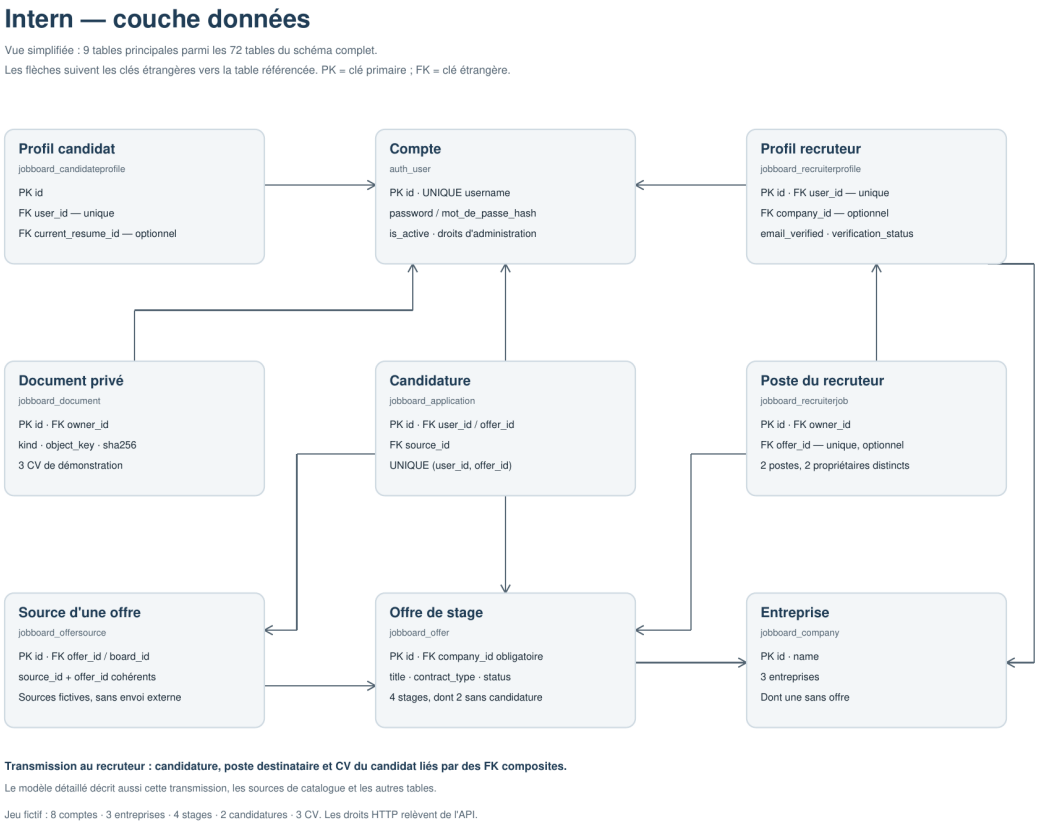


Figure 2. Vue des relations principales : comptes, profils, documents, candidatures, offres et entreprises. Les contraintes détaillées figurent dans le modèle et le schéma SQL.

Intégrité et confidentialité

Les clés étrangères garantissent les références ; les contraintes d'unicité empêchent les doublons. Des relations composites contrôlent la cohérence entre candidature, offre, destinataire et CV. Les mots de passe utilisent scrypt ; les empreintes SHA-256 des documents servent à identifier leurs contenus, et non à protéger les mots de passe.

La base ne connaît pas la session HTTP : elle protège les relations, tandis que l'API vérifie les autorisations du compte connecté. Un document privé ne peut donc pas être rendu public simplement parce que sa référence SQL existe.

Jeu de démonstration et vérification

Le jeu initial contient huit comptes, trois entreprises, quatre stages et deux candidatures, avec des cas vides et un compte en attente. Les documents de validation rapportent 34 contrôles SQL et 43 tests Django ciblés pour cette étape. Le jeu dédié aux mesures est ensuite complété par vingt candidats distincts.

Sources : donnees/MODELE.md ; donnees/schema.sql ; donnees/data.sql ; donnees/requetes.sql ; donnees/VALIDATION.md.

API, sessions et règles d’accès

L’API Python repose sur Django et Django REST Framework. Elle reçoit les requêtes du navigateur, valide les entrées, applique les règles d’accès et interroge PostgreSQL. Le client ne communique jamais directement avec la base.

1. Authentifier
CSRF + identifiants

→

2. Autoriser
Session + propriétaire

→

3. Consulter
SQL puis JSON

Authentification et accès aux ressources

Une session serveur matérialise la connexion. Le navigateur obtient un jeton CSRF, envoie les identifiants, puis reçoit la session et un jeton renouvelé. Un compte inactif ne peut pas se connecter. Les recherches de données privées sont limitées au propriétaire et les téléchargements de CV repassent par ce contrôle. Changer un identifiant dans l’URL ne doit pas permettre de lire un dossier tiers. Les permissions sont aussi appliquées aux recruteurs d’une même entreprise : l’appartenance commune ne remplace pas la propriété du poste ni une candidature explicite.

Contrôle	Comportement attendu
Inscription	Compte créé en attente ; aucune session active
Identifiants invalides	Échec de connexion sans divulguer les données privées
Compte inactif	Connexion refusée jusqu’à activation
Ressource tierce	Accès refusé ou ressource non divulguée
Publication recruteur	Vérifications d’adresse et d’entreprise exigées
CV transmis	Contrôle du destinataire et conservation des données partagées

Configuration et validation

Les paramètres de base, d’hôtes autorisés, de stockage et d’origine sont externalisés. La préparation des déploiements change les valeurs et les services, sans réécrire le parcours métier. Les exemples HTTP documentent les réponses attendues.

À l’étape d’intégration, les rapports annoncent 560 tests métier réussis, ainsi que les contrôles Django, migrations et analyse statique. Ce total appartient à la recette archivée ; il ne correspond pas à une nouvelle exécution réalisée lors de la rédaction.

Sources : metier/README.md ; metier/VALIDATION.md ; metier/api.http ; tests/README.md ; LIVRABLES.md, étape 6.

De l’interface au parcours complet

Le client React fournit les espaces candidat et recruteur ; l’administration utilise Django Admin. L’interface distingue erreurs, attente de validation et listes vides, afin de rendre les règles métier compréhensibles.

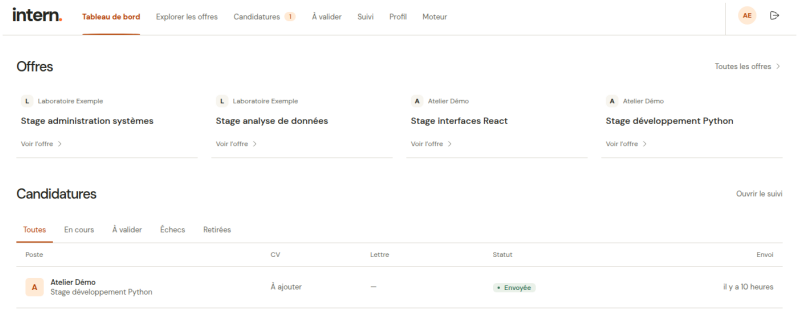


Figure 3. Application réelle sur les trois VM : offres fictives et suivi des candidatures, lors de la recette de l’étape 9.

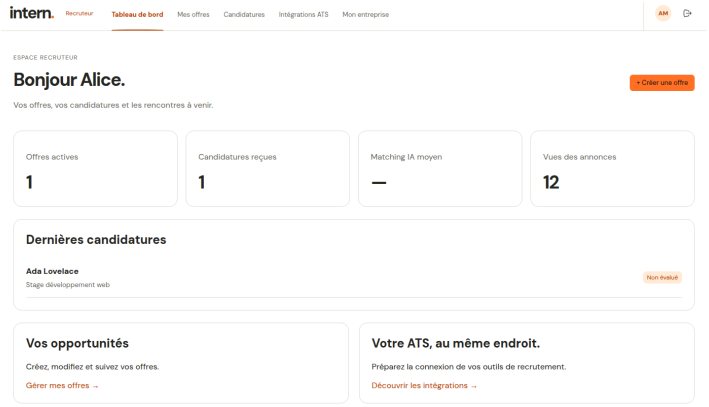


Figure 4. Espace recruteur issu d’un test Playwright avec réponses API contrôlées. Cette capture illustre l’interface ; elle ne prouve pas à elle seule une intégration complète.

La recette locale archivée couvre 15 parcours navigateur/API sur PostgreSQL éphémère. Le client compte 104 tests navigateur réussis avec réponses contrôlées ; trois scénarios optionnels de logos externes sont ignorés. La recette VM puis Docker comporte huit contrôles navigateur réels par déploiement.

À distinguer : T24, installation en suivant le README sur une VM vierge, et T25, comparaison visuelle au prototype, restent à vérifier dans le cahier de tests.

Sources : frontend/captures/README.md ; deploiement/docs/etape9.md ; tests/cahier_de_tests.html ; LIVRABLES.md.

Trois machines, un point d’entrée

Les trois couches sont installées sur Ubuntu Server 24.04.5 LTS. Des snapshots sont pris aux jalons de préparation, réseau et installation, afin de permettre un retour à un état connu.

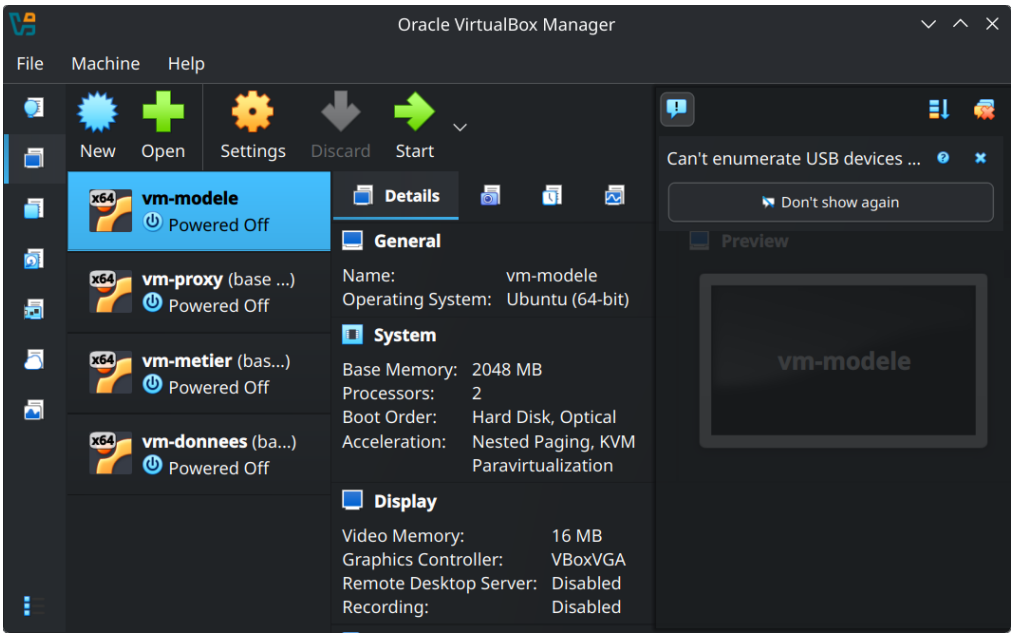


Figure 5. Gestionnaire VirtualBox : trois clones applicatifs et modèle commun. Les preuves CLI détaillent les ressources et les noms complets des snapshots.

Machine	Ressources	Adresse interne / service
vm-proxy	1 vCPU / 1 Gio	10.10.10.10 / Nginx :80
vm-metier	2 vCPU / 2 Gio	10.10.10.20 / Gunicorn :8000
vm-donnees	2 vCPU / 2 Gio	10.10.10.30 / PostgreSQL :5432

Réseau et vérification

Le réseau intnet-app utilise 10.10.10.0/24. Seul le proxy conserve un adaptateur NAT ; la redirection 127.0.0.1:8080 vers son port 80 donne accès à l’application. Les accès temporaires utilisés pour installer les paquets sont retirés après configuration.

Les six pings entre machines réussissent. Après redémarrage, les services démarrent automatiquement ; 34 contrôles SQL et huit contrôles navigateur passent. L’accès direct depuis l’hôte au métier et à la base échoue comme attendu. La matrice applicative complète figure page 10.

Trois exports OVA sont décrits avec tailles, empreintes et contrôles d’archive dans le manifeste. Les archives sont conservées hors dépôt : leur présence et leur accessibilité sur la machine de remise restent à vérifier.

Sources : deploiement/docs/etape7.md à etape10.md ; deploiement/reseau/README.md ; deploiement/docs/exports-etape10.json.

Composer les services et les réseaux

La composition reprend les trois fonctions : proxy Nginx, API Django/Gunicorn et PostgreSQL. Les images applicatives sont construites séparément ; le volume nommé pgdata conserve le stockage de la base après recréation des conteneurs.

Proxy
frontal / port hôte 8080

→

Métier
frontal + interne

→

Données
interne + volume pgdata

Mécanisme	Rôle dans le déploiement
Deux réseaux	Le proxy et la base ne partagent aucun réseau ; le métier relie les deux couches
Résolution par nom	Le métier utilise POSTGRES_HOST=donnees au lieu de localhost
Ports publiés	Seul le proxy publie 127.0.0.1:8080
Sondes de santé	La base doit être disponible avant le métier, puis le proxy
Images et exclusions	Bases légères, dépendances mises en cache, fichiers inutiles exclus
Persistance	Schéma et données importés sur volume vide ; contrôles après recréation

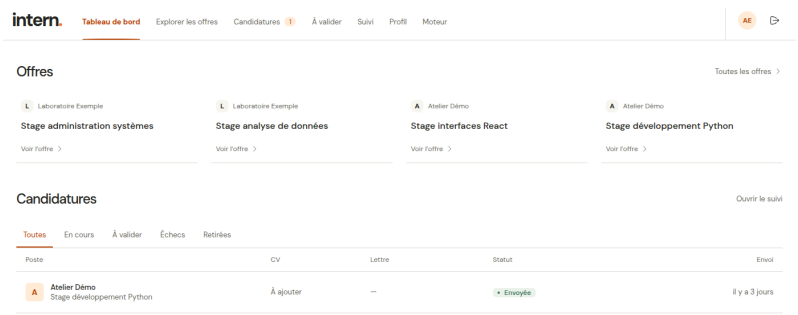


Figure 6. Application vérifiée sur Docker Engine à l'étape 14. Les preuves JSON, SQL et navigateur accompagnent cette capture.

Les préparations initiales incluent Podman ; la campagne comparative retenue utilise Docker Engine 28.3.3 rootless et Compose 2.39.4 sur Linux. Le moteur et le périmètre sont explicités pour éviter de confondre les mesures.

La vérification Docker rapporte trois services sains, neuf liaisons conformes, 34 contrôles SQL et huit contrôles navigateur réussis. La persistance est vérifiée après arrêt et recréation sans suppression du volume.

Sources : docker/compose.yaml ; docker/metier.Dockerfile ; docker/proxy.Dockerfile ; docker/docs/etape13.md et etape14.md.

Protocole et résultats de référence

Les mesures de référence sont réalisées les 12 et 14 octobre 2026 sur le même poste Fedora Linux 42 : Intel Core i5-1235U, 12 processeurs logiques et 15,297 Gio de RAM. VirtualBox 7.2.8 est comparé à Docker Engine 28.3.3 rootless, avec VPNKit et fuse-overlayfs, sans VM intermédiaire pour Docker.

Un protocole répété trois fois

Chaque série part des trois VM ou conteneurs arrêtés, déjà installés. Le démarrage se termine au premier catalogue HTTP 200 non vide. Après trente secondes au repos, la mémoire est relevée ; dix GET lisent complètement la réponse JSON du catalogue fictif. Une connexion HTTP neuve est utilisée à chaque appel. Le cache disque hôte est conservé.

Les médianes des trois séries sont retenues. La latence est la médiane des trois médianes de dix appels. Il s'agit d'une consultation du catalogue au repos, distincte du scénario authentifié JMeter présenté ensuite.

Grandeur	Trois VM	Docker	Écart Docker / VM
Démarrage	20,473 s	49,844 s	+143,5 %
Mémoire RSS hôte	1,454 Gio	0,568 Gio	-60,9 %
Disque : fichiers	10,783 Gio	0,809 Gio	-92,5 %
Disque : blocs alloués	10,755 Gio	0,864 Gio	-92,0 %
Latence de consultation	62,431 ms	83,385 ms	+33,6 %
Mise en place humaine	Non chronométrée	Non chronométrée	Non calculable

Périmètre des ressources

La RSS VM additionne les processus des trois machines, hors VBoxSVC. Docker inclut conteneurs, daemon, containerd, shims, FUSE, réseau et publication rootless. La RSS peut compter plusieurs fois des pages partagées ; elle ne représente pas une quantité de mémoire physique exclusive.

Le disque VM inclut les snapshots. Docker compte les fichiers uniques des images, les couches modifiables et le volume de données, sans additionner plusieurs fois les couches communes. Les caches de construction et les OVA sont exclus. La taille des fichiers et les blocs réellement alloués sont deux mesures différentes.

1. De combien les résultats diffèrent-ils ?

Dans cette configuration, Docker réduit la mémoire et le stockage mesurés, mais atteint le catalogue 29,371 s plus tard et ajoute 20,955 ms à la latence médiane de consultation. Ces écarts décrivent un poste, un protocole et des versions précises ; ils ne désignent pas de gagnant universel.

Sources : docs/comparaison.md ; deploiement/docs/mesures-etape10.json ; docker/docs/mesures-etape14.json.

Isolation, explications et contreparties

Matrice de joignabilité applicative

Origine	Proxy :80	Métier :8000	Données :5432
Hôte	Oui via 8080	Refus attendu	Refus attendu
Proxy	Sans objet	Oui	Refus attendu
Métier	Oui	Sans objet	Oui
Données	Refus attendu	Oui	Sans objet

Les neuf liaisons testées sont conformes sur chaque déploiement. Les services sont démarrés pendant les contrôles négatifs. Les preuves Docker testent les IP réelles, afin qu'un refus ne soit pas simplement dû à l'absence d'un nom DNS. Cette matrice décrit des accès TCP observés, pas toutes les permissions applicatives.

2. Quelles propriétés expliquent les écarts ?

Les conteneurs partagent le noyau de l'hôte et évitent trois systèmes invités et leurs services. Les couches d'images sont mutualisées. Cela explique une partie de l'économie de mémoire et de stockage ; les snapshots VM contribuent aussi à l'écart disque.

Le délai de démarrage Docker peut être lié aux sondes de santé et à l'ordre des dépendances. Le stockage FUSE et le réseau rootless sont d'autres facteurs plausibles. Leur contribution individuelle n'a pas été isolée : ces explications restent des hypothèses compatibles avec les mesures.

Les ressources ne sont pas strictement égalisées : les VM disposent de 1/2/2 vCPU et de 1/2/2 Gio, tandis que les conteneurs n'ont pas de quotas équivalents. Python passe de 3.12.3 à 3.12.15 et Nginx de 1.24.0 à 1.28.0 ; un service de logos diffère aussi dans la référence initiale. On ne peut donc pas attribuer tout l'écart à la seule isolation.

3. Que perd-on avec les conteneurs ?

Les VM possèdent un noyau invité séparé derrière l'hyperviseur. Les conteneurs reposent sur les mécanismes du noyau hôte ; une défaillance de ce noyau peut affecter toute la pile. Le mode rootless réduit les privilèges du moteur, sans créer la même frontière qu'une VM.

Un conteneur Linux n'embarque pas un noyau indépendant et ne permet pas de substituer librement un autre système. Les images facilitent la reconstruction du logiciel, mais ne remplacent ni les sauvegardes du volume PostgreSQL ni les exports et snapshots de VM.

Point manquant : la durée humaine de mise en place n'a pas été chronométrée. Elle ne peut pas être déduite des temps de démarrage ; un nouvel essai depuis zéro serait nécessaire pour compléter cette ligne.

Sources : docs/comparaison.md ; deploiement/docs/joignabilite-etape10.json ; docker/docs/joignabilite-etape14.json.

Tester un parcours authentifié

Apache JMeter 5.6.3, avec Java 21, reproduit la préparation CSRF, la connexion et la consultation des candidatures personnelles. Chaque candidat dispose d'un compte fictif distinct. Les cookies sont conservés et les assertions contrôlent le code HTTP ainsi que le contenu attendu.

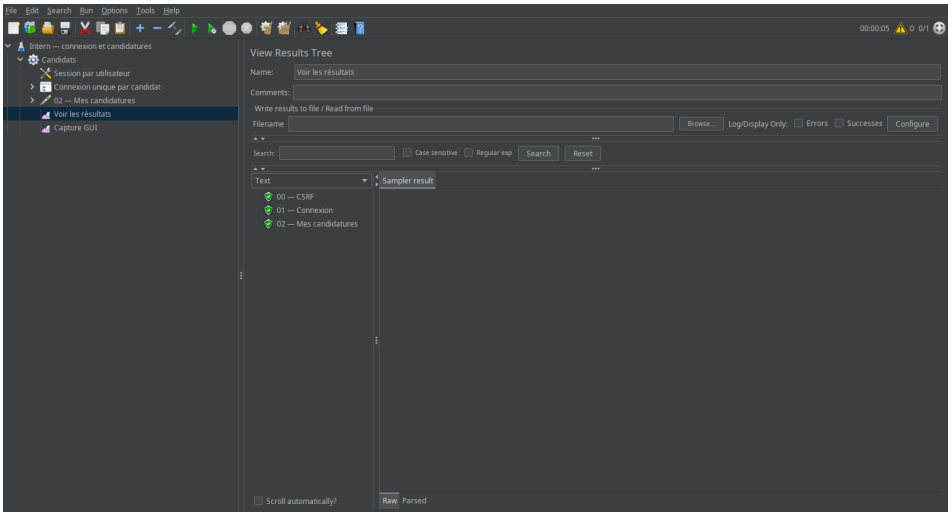


Figure 7. Débogage graphique : un utilisateur, une itération ; préparation CSRF, connexion et consultation réussissent.

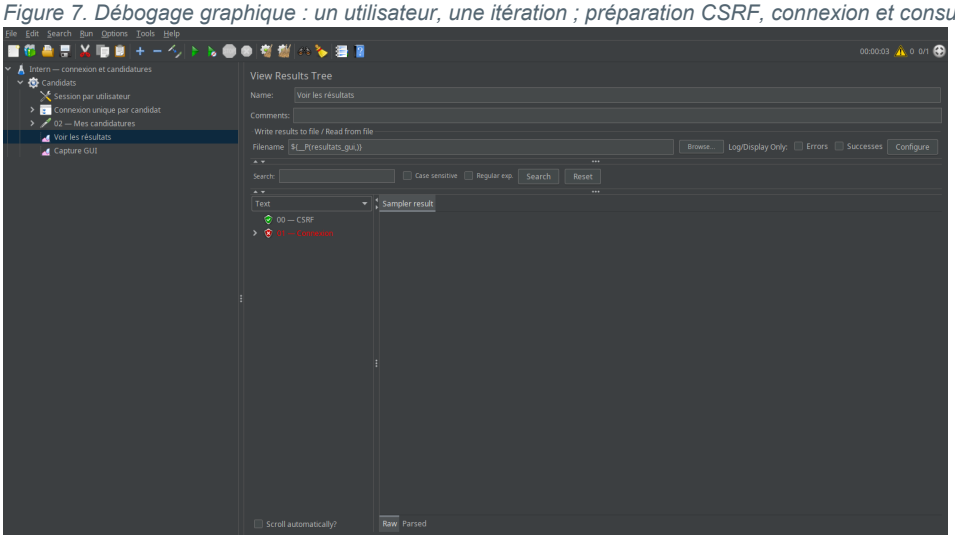


Figure 8. Contrôle négatif : mot de passe incorrect, HTTP 401 et assertion en échec ; la consultation n'est pas exécutée.

Un scénario qui détecte ses propres échecs

Une réponse HTTP 200 ne suffit pas : le contenu doit correspondre au candidat connecté et contenir la candidature attendue. Le contrôle négatif vérifie que le plan signale effectivement un refus d'authentification. Un essai complémentaire avec deux candidats vérifie l'isolation des sessions.

L'interface graphique sert à construire et déboguer le plan. Les campagnes de performance sont exécutées en ligne de commande depuis l'hôte, hors du système mesuré. La base de charge est dédiée ; aucune candidature réelle ni aucun envoi externe ne font partie du scénario.

Sources : mesure/plan.jmx ; mesure/ETAPE15.md ; mesure/resultats/etape15/ ; mesure/comptes.csv.

Vérifier l’instrument de mesure

Avant de comparer les infrastructures, une ressource statique du proxy est sollicitée sans temporisation ni accès à la base. Le débit observé caractérise le chemin injecteur/réseau/proxy ; il ne constitue pas une limite intrinsèque de JMeter.

Charge statique	Débit réussi	p95	Erreurs
1 utilisateur	587,30 req/s	3 ms	0 %
20 utilisateurs	3 250,15 req/s	12 ms	0 %

À vingt utilisateurs et une seconde de réflexion, le profil applicatif ne peut dépasser environ vingt consultations/s. Le chemin statique fournit donc une marge observée d’environ 162,5 fois. La forte charge de l’hôte pendant l’essai statique justifie l’arrêt de cette progression. Une extension du profil nécessite un nouvel étalonnage.

Paramètres communs aux campagnes

Paramètre	Valeur retenue
Utilisateurs simultanés	1, 5, 10, 20 candidats distincts
Montée / stabilisation	30 s / 15 s
Fenêtre de mesure stable	120 s ; durée programmée de 170 s
Temps de réflexion	1 000 ms entre consultations
Connexion	Une seule connexion par candidat
Repos minimal entre paliers	900 s pour préserver les protections de connexion

Répétabilité : deux contrôles à dix candidats

Exécution	Consultations stables	Débit	p95
Contrôle 1	1 176	9,800 req/s	22 ms
Contrôle 2	1 175	9,792 req/s	21 ms

L’écart symétrique est de 0,09 % pour le débit et de 4,65 % pour le p95, sans erreur. Ces deux grandeurs restent sous le seuil de 10 % retenu pour ce contrôle. Le p99 varie de 19 %, ce qui rappelle la variabilité des valeurs de queue.

Pendant les contrôles applicatifs, le CPU injecteur moyen reste proche de 4,8 % d’un cœur. L’étalonnage soutient donc la mesure dans la plage prévue ; il ne valide pas une charge arbitrairement supérieure.

Sources : mesure/ETAPE16.md ; mesure/protocole.json ; mesure/resultats/etape16/etalonnage.json.

Comparer débit et latence

Deux séries inversent l'ordre de passage : VM puis Docker, ensuite Docker puis VM. Un seul déploiement est actif ; le même export SQL est restauré dans les bases dédiées avant chaque campagne. Les protections de connexion restent actives. CPU, mémoire, I/O et connexions PostgreSQL sont relevés en parallèle.

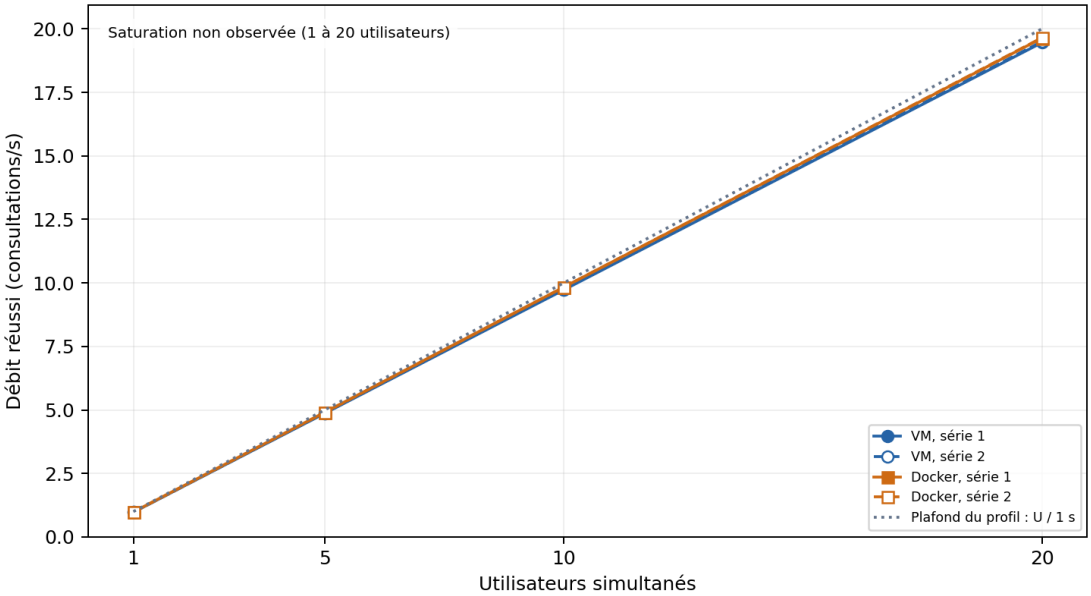


Figure 9. Débit des consultations réussies selon le nombre d'utilisateurs. Le débit continue à augmenter ; la limite du profil avec réflexion ne doit pas être confondue avec une capacité serveur.

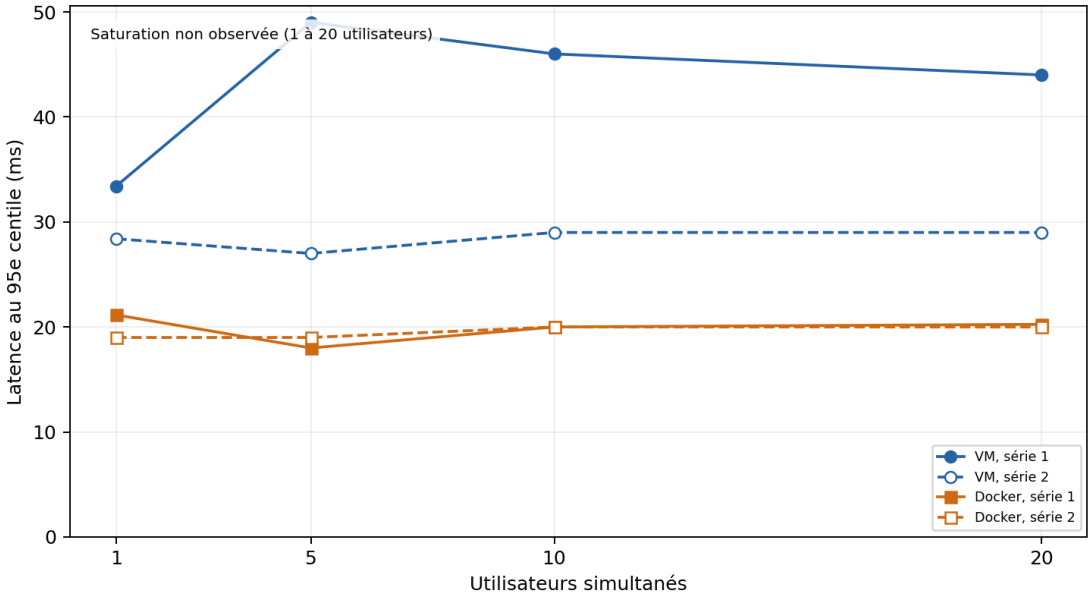


Figure 10. Latence p95, deux séries conservées séparément. Docker présente un p95 plus faible dans la plage étudiée ; la variabilité entre passages VM reste visible.

La collecte comprend seize paliers et 48 relevés agrégés par service et palier, accompagnés des fichiers détaillés. Les statistiques portent sur les fenêtres stables, sans mélanger montée, stabilisation et plateau.

Sources : mesure/ETAPE17.md ; mesure/docs/rapport-charge.md ; mesure/resultats/etape17/ ; courbes de l'étape 18.

Des résultats, avec leurs limites

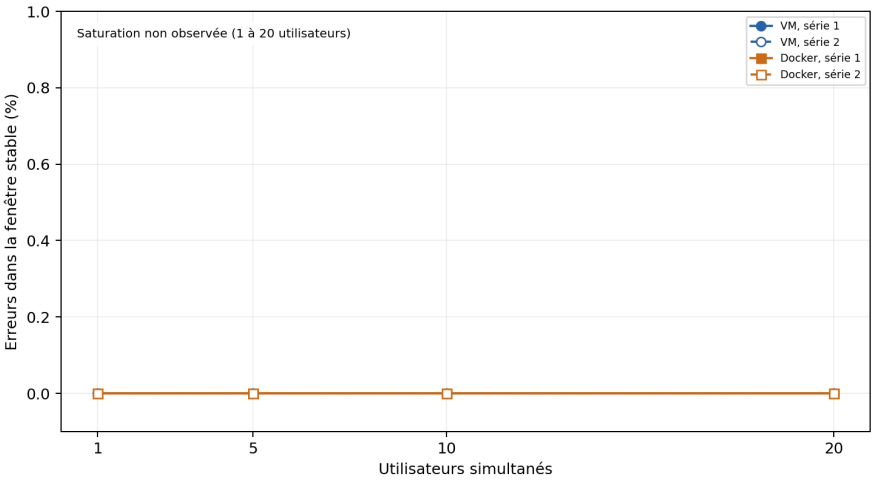


Figure 11. Taux d'erreur nul dans les fenêtres analysées ; les quatre courbes se superposent à zéro.

À vingt utilisateurs	Trois VM	Docker
Débit moyen des deux séries	19,479 req/s	19,642 req/s
p95, séries 1 / 2	44 / 29 ms	20,25 / 20 ms
CPU métier, séries 1 / 2	27,92 / 26,26 %	31,67 / 31,16 %
Saturation / goulot	Non déterminés	Non déterminés

Le débit moyen diffère de +0,83 % en faveur de Docker. Le p95 baisse de 53,98 % dans la première série et de 31,03 % dans la seconde. Les 22 199 échantillons bruts ne présentent aucune erreur ; 16 902 consultations appartiennent aux fenêtres stables.

Pourquoi les débits restent proches

Chaque utilisateur attend une seconde entre deux consultations. Avec U utilisateurs et un temps de réponse R en secondes, le débit est approximativement U/(1 + R). À vingt utilisateurs, le profil plafonne autour de vingt consultations/s, même si le serveur pourrait en traiter davantage. Le maximum observé n'est donc pas une capacité maximale.

Saturation et interprétation des métriques

Aucun plateau de débit accompagné d'une hausse durable du p95 n'est observé. Le métier est le service le plus chargé en CPU, sans saturation démontrée. Les VM utilisent des compteurs de processus invités et Docker cgroup v2 ; ces périmètres ne mesurent pas un coût CPU total strictement comparable.

Un arrêt du moteur Docker a interrompu la transition après la première campagne VM. Les paliers ont été conservés puis la collecte reprise. Cette interruption, la variation de charge de l'hôte et les différences de ressources limitent l'interprétation de l'ordre de passage et des causes de latence. La mesure couvre un seul poste et un parcours de lecture, pas une charge de production variée.

Sources : mesure/docs/rapport-charge.md ; mesure/ETAPE17.md ; mesure/ETAPE18.md ; mesure/resultats/etape18/analyse.json.

Travail réalisé et finalisation

Le projet relie conception fonctionnelle, intégrité relationnelle, API et interface à deux déploiements reproductibles. Les mesures montrent une économie de ressources pour les conteneurs dans la configuration de référence et un p95 plus faible sous le profil étudié. Elles ne permettent pas d'annoncer une capacité maximale ni un goulot démontré.

Points à compléter avant la remise

Priorité	Action et preuve attendue
Périmètre	Confirmer l'acceptation de l'adaptation Intern et la validation du jalon Figma.
Prototype	Vérifier l'accès lecture et comparer les parcours réels au prototype ; renseigner T25.
Installation	Rejouer le README sur une VM vierge et conserver le résultat de T24.
Charge	Préparer davantage de comptes, réétalonner et prolonger les paliers jusqu'à saturation ; actualiser courbes et analyse.
Comparabilité	Rejouer les séries alternées sans interruption et documenter les ressources et versions.
Temps humain	Si cette comparaison est complétée, chronométrer un nouvel essai depuis zéro ; ne pas reconstituer les temps initiaux.
Remise	Vérifier les trois OVA hors dépôt, finaliser les fichiers Git et préparer la démonstration.

Organisation du dossier de preuves

Conception : analyse/ et maquette/. **Application :** donnees/, metier/ et frontend/. **Recette :** tests/. **Infrastructure :** deploiement/ et docker/. **Mesures :** mesure/plan.jmx, protocole.json, resultats/ et docs/. LIVRABLES.md reste le point d'entrée vers les fichiers et les états de vérification.

Les PDF spécialisés restent disponibles : analyse/cas_d_usage.pdf, tests/cahier_de_tests.pdf, docs/compte_rendu_comparaison.pdf et mesure/docs/rapport-charge.pdf. Les scripts SQL, configurations, résultats bruts et OVA restent des livrables distincts du présent compte rendu.

Statut de cette version

Ce rapport consolide les résultats archivés disponibles au 16 octobre 2026. Les captures proviennent du dépôt et sont légendées selon leur contexte. La rédaction ne relance ni les déploiements ni les essais et n'atteste pas l'état actuel des services. Les points ouverts restent explicitement signalés pour guider la finalisation.

Sources : LIVRABLES.md ; tests/cahier_de_tests.html, T24-T25 ; docs/consignes/ ; rapports techniques cités dans les pages précédentes.